TP Pratique : Decouverte de Docker pour le DevOps

Enseignant DevOps

11 mars 2026

Objectifs du TP :

- Comprendre les concepts de base de la conteneurisation
- Installer et configurer Docker
- Manipuler des images et des conteneurs
- Creer sa propre image avec un Dockerfile
- Deployer une application Python conteneurisee

Table des matières

1 Introduction au DevOps **3**
1.1 Qu'est-ce que le DevOps ? . . . 3
1.2 Pourquoi automatiser le deploiement ? . . . 3
1.3 ROle de la conteneurisation . . . 3

2 Introduction a la conteneurisation **3**
2.1 Definition d'un conteneur . . . 3
2.2 Difference entre machine virtuelle et conteneur . . . 3
2.3 Presentation de Docker . . . 4

3 Installation de Docker **4**
3.1 Installation sur Linux (Ubuntu/Debian) . . . 4
3.2 Verification de l'installation . . . 4
3.3 Premiere commande Docker . . . 5

4 Manipulation des images Docker **5**
4.1 Commande docker pull . . . 5
4.2 Commande docker images . . . 5
4.3 Commande docker search . . . 6
4.4 Commande docker rmi . . . 6

5 Creation d'un conteneur **6**
5.1 Commande docker run . . . 6
5.2 Commande docker ps . . . 7
5.3 Commande docker stop . . . 7
5.4 Commande docker start . . . 7
5.5 Commande docker rm . . . 8
5.6 Exemple complet : Lancer Nginx . . . 8

6 Creation d'une image personnalisee **9**
6.1 Qu'est-ce qu'une image Docker ? 9
6.2 Pourquoi creer sa propre image ? 9

7 Creation d'un Dockerfile **9**
7.1 Structure d'un Dockerfile 9
7.2 Exemple complet : Application Python Flask 9
7.2.1 Structure du projet 9
7.2.2 Code de l'application (app.py) 10
7.2.3 Fichier de dependances (requirements.txt) 10
7.2.4 Dockerfile 10

8 Construction de l'image **11**
8.1 Commande docker build 11
8.2 Commande docker tag 11

9 Lancement du conteneur a partir de l'image **11**
9.1 Lancement avec mapping de ports 11

10 TP Pratique guide **12**
10.1 Etape 1 : Preparation de l'environnement 12
10.2 Etape 2 : Creation de l'application 12
10.3 Etape 3 : Fichier des dependances 13
10.4 Etape 4 : Creation du Dockerfile 13
10.5 Etape 5 : Construction de l'image 13
10.6 Etape 6 : Lancement du conteneur 13
10.7 Etape 7 : Verification 14
10.8 Etape 8 : Arret et nettoyage 14

11 Exercices supplementaires **14**
11.1 Exercice 1 : Application Redis 14
11.2 Exercice 2 : Modification du Dockerfile 15
11.3 Exercice 3 : Nettoyage complet 15

12 Conclusion **16**
12.1 Resume du TP 16
12.2 Avantages de la conteneurisation dans le DevOps 16

1 Introduction au DevOps

1.1 Qu'est-ce que le DevOps ?

Le **DevOps** est un ensemble de pratiques qui vise a unifier le developpement logiciel (Dev) et les operations informatiques (Ops). L'objectif principal est de raccourcir le cycle de vie du developpement tout en livrant des fonctionnalites, des correctifs et des mises a jour frequemment, en etroite alignement avec les objectifs metier.

1.2 Pourquoi automatiser le deploiement ?

L'automatisation du deploiement presente plusieurs avantages :

- **Reproductibilite** : L'environnement de production est identique a l'environnement de developpement
- **Rapidite** : Les deploiements sont plus rapides et moins sujets aux erreurs humaines
- **Fiabilite** : Moins d'erreurs de configuration
- **Evolution** : Facilite la mise a l'echelle des applications

1.3 ROle de la conteneurisation

La conteneurisation est devenue un pilier du DevOps car elle permet d'emballer une application avec toutes ses dependances dans un environnement isole et portable. Cela resout le celebre probleme "ca marche sur ma machine".

2 Introduction a la conteneurisation

2.1 Definition d'un conteneur

Definition

Un **conteneur** est une unite standard de logiciel qui empaquette le code et toutes ses dependances pour que l'application s'execute rapidement et de maniere fiable d'un environnement informatique a un autre.

2.2 Difference entre machine virtuelle et conteneur

Machine Virtuelle	Conteneur
Inclut son propre systeme d'exploitation	Partage le noyau de l'hote
Demarrage lent (plusieurs minutes)	Demarrage rapide (secondes)
Taille importante (Go)	Taille legere (Mo)
Isolation forte	Isolation au niveau processus
Consommation memoire elevee	Consommation memoire faible

2.3 Presentation de Docker

Docker est la plateforme de conteneurisation la plus populaire. Elle permet de :

- Creer des images (modeles en lecture seule)
- Distribuer ces images via des registres (Docker Hub)
- Executer des conteneurs a partir de ces images

3 Installation de Docker

3.1 Installation sur Linux (Ubuntu/Debian)

```
# Mise a jour des paquets
sudo apt update

# Installation des prerequis
sudo apt install apt-transport-https ca-certificates curl software-properties-common

# Ajout de la cle GPG officielle de Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# Ajout du depot Docker
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Installation de Docker
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io

# Demarrage du service Docker
sudo systemctl start docker
sudo systemctl enable docker

# Ajout de l'utilisateur courant au groupe docker (pour eviter sudo)
sudo usermod -aG docker $USER

# IMPORTANT : Deconnectez-vous et reconnectez-vous pour que les changements prennent effet
```

Listing 1 – Installation de Docker sur Ubuntu

3.2 Verification de l'installation

```
# Verifier la version de Docker
docker --version

```

```
# Verifier que le service tourne
systemctl status docker

# Tester Docker sans sudo (apres reconnexion)
docker version
```

Listing 2 – Verifier que Docker est bien installe

3.3 Premiere commande Docker

```
# Lancer le conteneur "hello-world" pour tester
docker run hello-world

# Cette commande va :
# 1. Chercher l'image 'hello-world' localement
# 2. La telecharger depuis Docker Hub si absente
# 3. Creer et demarrer un conteneur
# 4. Afficher un message de bienvenue
```

Listing 3 – Premier conteneur

4 Manipulation des images Docker

4.1 Commande docker pull

```
# Syntaxe : docker pull <nom_image>:<tag>
# Le tag "latest" est utilise par defaut

# Telecharger la derniere version de nginx
docker pull nginx

# Telecharger une version specifique d'Ubuntu
docker pull ubuntu:22.04

# Telecharger l'image Python officielle
docker pull python:3.9-slim

# Explication : Ces images sont stockees localement pour etre
   utilisees plus tard
```

Listing 4 – Telecharger des images

4.2 Commande docker images

```
# Afficher toutes les images telechargees
docker images

# Afficher avec plus de details
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"

```

```
# Explication : Cette commande montre :
# - Le depot (nom de l'image)
# - Le tag (version)
# - L'ID de l'image
# - La date de creation
# - La taille
```

Listing 5 – Lister les images disponibles

4.3 Commande docker search

```
# Rechercher des images liees a Python
docker search python

# Rechercher avec des filtres
docker search --filter stars=1000 --filter is-official=true nginx

# Explication : Cette commande interroge Docker Hub pour trouver
   des images publiques
```

Listing 6 – Chercher des images sur Docker Hub

4.4 Commande docker rmi

```
# Supprimer une image specifique
docker rmi nginx

# Supprimer plusieurs images
docker rmi ubuntu:22.04 python:3.9-slim

# Supprimer toutes les images non utilisees
docker image prune

# Explication : On ne peut supprimer une image que si aucun
   conteneur ne l'utilise
```

Listing 7 – Supprimer des images

5 Creation d'un conteneur

5.1 Commande docker run

```
# Lancer un conteneur Nginx en mode interactif
docker run nginx
# ^ Cette commande bloque le terminal (mode attache)

# Lancer en mode detache (arriere-plan)
docker run -d nginx

# Lancer avec un nom personnalise et mapping de ports
```

```
docker run -d --name mon_nginx -p 8080:80 nginx
# Explication : -p 8080:80 mappe le port 8080 de l'hote vers le port 80 du conteneur

# Lancer un conteneur interactif avec shell
docker run -it ubuntu bash
# -i : interactif, -t : pseudo-terminal

# Lancer et supprimer automatiquement apres arret
docker run --rm hello-world
```

Listing 8 – Lancer des conteneurs

5.2 Commande docker ps

```
# Afficher les conteneurs en cours d'execution
docker ps

# Afficher tous les conteneurs (meme arretes)
docker ps -a

# Afficher avec format personnalise
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Explication : Utile pour voir quels conteneurs tournent et leurs IDs
```

Listing 9 – Lister les conteneurs

5.3 Commande docker stop

```
# Arreter un conteneur par son nom
docker stop mon_nginx

# Arreter un conteneur par son ID
docker stop 1a2b3c4d5e6f

# Arreter plusieurs conteneurs
docker stop conteneur1 conteneur2

# Explication : Envoie un signal SIGTERM au processus principal du conteneur
```

Listing 10 – Arreter des conteneurs

5.4 Commande docker start

```
# Demarrer un conteneur existant
docker start mon_nginx

```

```
# Demarrer en mode attache
docker start -a mon_nginx

# Explication : Reutilise un conteneur deja cree sans en recreer un
    nouveau
```

Listing 11 – Demarrer un conteneur arrete

5.5 Commande docker rm

```
# Supprimer un conteneur arrete
docker rm mon_nginx

# Forcer la suppression d'un conteneur en cours d'execution
docker rm -f mon_nginx

# Supprimer plusieurs conteneurs
docker rm conteneur1 conteneur2 conteneur3

# Supprimer tous les conteneurs arretes
docker container prune

# Explication : Les conteneurs doivent etre arretes (sauf avec -f)
```

Listing 12 – Supprimer des conteneurs

5.6 Exemple complet : Lancer Nginx

```
# 1. Telecharger l'image Nginx
docker pull nginx

# 2. Lancer un conteneur Nginx
docker run -d --name serveur_web -p 8080:80 nginx

# 3. Verifier que le conteneur tourne
docker ps

# 4. Tester dans le navigateur : http://localhost:8080

# 5. Voir les logs
docker logs serveur_web

# 6. Arreter le conteneur
docker stop serveur_web

# 7. Le supprimer
docker rm serveur_web
```

Listing 13 – Lancer un serveur web Nginx

6 Creation d'une image personnalisee

6.1 Qu'est-ce qu'une image Docker ?

Une image Docker est un modele en lecture seule contenant les instructions pour creer un conteneur. Elle est construite a partir d'un fichier appele **Dockerfile**.

6.2 Pourquoi creer sa propre image ?

- Personnaliser l'environnement selon les besoins de votre application
- Embarquer votre code source
- Installer des dependances specifiques
- Garantir la reproductibilite

7 Creation d'un Dockerfile

7.1 Structure d'un Dockerfile

```
# FROM : Image de base
FROM python:3.9-slim

# WORKDIR : Definit le repertoire de travail
WORKDIR /app

# COPY : Copie des fichiers de l'hote vers le conteneur
COPY requirements.txt .
COPY app.py .

# RUN : Execute des commandes pendant la construction
RUN pip install --no-cache-dir -r requirements.txt

# EXPOSE : Informe que le conteneur ecoute sur ce port
EXPOSE 5000

# CMD : Commande executee au demarrage du conteneur
CMD ["python", "app.py"]
```

Listing 14 – Instructions de base d'un Dockerfile

7.2 Exemple complet : Application Python Flask

Creons une application web simple avec Flask.

7.2.1 Structure du projet

```
mon_app_flask/
    app.py
    requirements.txt
    Dockerfile
```

Listing 15 – Arborescence du projet

7.2.2 Code de l'application (app.py)

```python
from flask import Flask
import socket

app = Flask(__name__)

@app.route('/')
def hello():
    return """
    <html>
        <head><title>Ma super app</title></head>
        <body>
            <h1>Bienvenue dans mon conteneur Docker !</h1>
            <p>Cette application tourne dans un conteneur.</p>
            <p>Hostname: <strong>{}</strong></p>
        </body>
    </html>
    """.format(socket.gethostname())

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Listing 16 – Application Flask simple

7.2.3 Fichier de dependances (requirements.txt)

```
Flask==2.3.2
```

Listing 17 – Demarches Python

7.2.4 Dockerfile

```dockerfile
# Utiliser l'image officielle Python comme base
FROM python:3.9-slim

# Definir le maintainer (optionnel)
LABEL maintainer="etudiant@example.com"

# Definir le repertoire de travail dans le conteneur
WORKDIR /app

# Copier d'abord le fichier des dependances (optimisation du cache)
COPY requirements.txt .

# Installer les dependances
RUN pip install --no-cache-dir -r requirements.txt

# Copier le reste du code
COPY app.py .

```

```
# Informer que l'application utilise le port 5000
EXPOSE 5000

# Commande pour lancer l'application
CMD ["python", "app.py"]
```

Listing 18 – Dockerfile pour l'application Flask

8 Construction de l'image

8.1 Commande docker build

```
# Se placer dans le dossier contenant le Dockerfile
cd mon_app_flask

# Construire l'image avec un tag
docker build -t ma_flask_app:v1 .

# Explication :
# -t : tag (nom:version)
# .  : contexte de build (dossier courant)

# Options utiles :
docker build --no-cache -t ma_flask_app:v2 .
# --no-cache : ignore le cache et reconstruit tout

# Verifier que l'image a ete creee
docker images | grep ma_flask_app
```

Listing 19 – Construire une image

8.2 Commande docker tag

```
# Donner un nouveau tag a une image existante
docker tag ma_flask_app:v1 mon_user/ma_flask_app:latest

# Pour preparer le push vers Docker Hub
docker tag ma_flask_app:v1 mon_user/ma_flask_app:v1

# Explication : Utile pour versionner ou pousser vers un registre
```

Listing 20 – Tagger une image

9 Lancement du conteneur a partir de l'image

9.1 Lancement avec mapping de ports

```
# Lancer le conteneur en mode detache
docker run -d --name mon_app -p 5000:5000 ma_flask_app:v1

# Verifier que le conteneur tourne
docker ps

# Tester l'application : http://localhost:5000

# Voir les logs
docker logs mon_app

# Options supplementaires :
# -d : mode detache (arriere-plan)
# --name : donne un nom au conteneur
# -p : mapping de ports (port_hote:port_conteneur)

# Si le port 5000 est deja utilise :
docker run -d --name mon_app -p 8080:5000 ma_flask_app:v1
# Acces : http://localhost:8080
```

Listing 21 – Lancer le conteneur Flask

10 TP Pratique guide

Enonce du TP

Vous allez creer une application web simple avec Python Flask, la conteneuriser avec Docker, et la deployer localement.

10.1 Etape 1 : Preparation de l'environnement

```
mkdir ~/mon_tp_docker
cd ~/mon_tp_docker
```

Listing 22 – Creer le dossier du projet

10.2 Etape 2 : Creation de l'application

Creez un fichier `app.py` :

```
from flask import Flask
import datetime
import socket
import sys
import os

app = Flask(__name__)

@app.route('/')
def accueil():
```

```
    now = datetime.datetime.now()
    return f"""
    <h1>TP Docker - Application Flask</h1>
    <p>Date et heure actuelles : {now.strftime('%d/%m/%Y %H:%M:%S')}</p>
    <p>Le conteneur fonctionne correctement !</p>
    """

@app.route('/info')
def info():
    return f"""
    <h2>Informations du conteneur</h2>
    <ul>
        <li>Hostname : {socket.gethostname()}</li>
        <li>Plateforme : {os.uname().sysname}</li>
        <li>Python version : {sys.version}</li>
    </ul>
    """

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Listing 23 – Application a creer

10.3 Etape 3 : Fichier des dependances

Creez `requirements.txt` :

```
Flask==2.3.2
```

10.4 Etape 4 : Creation du Dockerfile

Creez `Dockerfile` :

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

10.5 Etape 5 : Construction de l'image

```
docker build -t tp_flask_app .
```

10.6 Etape 6 : Lancement du conteneur

```
docker run -d --name tp_conteneur -p 5000:5000 tp_flask_app
```

10.7 Etape 7 : Verification

```
# Voir les conteneurs actifs
docker ps

# Voir les logs
docker logs tp_conteneur

# Tester dans le navigateur : http://localhost:5000
# et http://localhost:5000/info
```

10.8 Etape 8 : Arret et nettoyage

```
# Arreter le conteneur
docker stop tp_conteneur

# Supprimer le conteneur
docker rm tp_conteneur

# Optionnel : supprimer l'image
docker rmi tp_flask_app
```

11 Exercices supplementaires

11.1 Exercice 1 : Application Redis

Exercice 1

Creez un conteneur Redis et connectez-vous a son interface en ligne de commande.

Objectifs :

- Utiliser `docker pull` pour telecharger l'image Redis
- Lancer un conteneur Redis en mode detache
- Utiliser `docker exec` pour ouvrir un shell Redis

```
# Telecharger l'image Redis
docker pull redis:alpine

# Lancer Redis en mode detache
docker run -d --name mon_redis -p 6379:6379 redis:alpine

# Se connecter au conteneur et lancer redis-cli
docker exec -it mon_redis redis-cli

# Tester quelques commandes Redis
# > set ma_cle "Bonjour"
# > get ma_cle
# > exit

# Nettoyer
```

```
docker stop mon_redis
docker rm mon_redis
```

Listing 24 – Correction

11.2 Exercice 2 : Modification du Dockerfile

Exercice 2

Modifiez le Dockerfile de l'application Flask pour ajouter une variable d'environnement et changer le port par defaut.

Indices :
- Utilisez `ENV` pour definir une variable
- Modifiez le port dans l'application

```
# Dockerfile modifie
FROM python:3.9-slim

# Ajout d'une variable d'environnement
ENV APP_ENV=production
ENV APP_PORT=8080

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .

EXPOSE 8080
CMD ["python", "app.py"]

# Construction et lancement
docker build -t flask_enhanced .
docker run -d -p 8080:8080 flask_enhanced
```

Listing 25 – Correction

11.3 Exercice 3 : Nettoyage complet

Exercice 3

Nettoyez votre systeme Docker : supprimez tous les conteneurs arretes et toutes les images non utilisees.

```
# Supprimer tous les conteneurs arretes
docker container prune -f

# Supprimer toutes les images non utilisees
docker image prune -a -f

# Supprimer les volumes non utilises
docker volume prune -f
```

```

# Supprimer les reseaux non utilises
docker network prune -f

# Tout nettoyer d'un coup (attention !)
docker system prune -a -f --volumes

# Verifier qu'il ne reste rien
docker ps -a
docker images
```

Listing 26 – Correction

12 Conclusion

12.1 Resume du TP

Au cours de ce TP, nous avons :

- Compris les concepts fondamentaux de Docker et de la conteneurisation
- Installe et configure Docker sur notre machine
- Manipule des images et des conteneurs
- Cree notre propre image avec un Dockerfile
- Deploye une application Flask conteneurisee
- Realise des exercices pratiques pour approfondir

12.2 Avantages de la conteneurisation dans le DevOps

La conteneurisation avec Docker apporte des benefices considerables :

- **Portabilite** : Une application conteneurisee tourne partout
- **Isolation** : Les applications n'interferent pas entre elles
- **Efficacite** : Utilisation optimale des ressources
- **Reproductibilite** : Meme environnement partout
- **CICD** : Integration facile dans les pipelines
- **Mise a l'echelle** : Facilite l'orchestration (Kubernetes, Swarm)

Pour aller plus loin

Ce TP vous a donne les bases. Pour approfondir, explorez :

- Docker Compose pour les applications multi-conteneurs
- Les reseaux Docker
- Les volumes pour la persistance des donnees
- Kubernetes pour l'orchestration a grande echelle